

KUBERNETES

Complete Architecture & Technical Deep-Dive

Storage Nodes • Worker Nodes • Containers • Pods • Control Plane

This document provides a complete architectural overview of Kubernetes, covering the Control Plane, Worker Nodes, Storage Nodes, Containers, and Pods. It is designed for engineers, architects, and developers seeking a thorough understanding of how Kubernetes orchestrates containerized workloads at scale.

Table of Contents

- 1. Introduction to Kubernetes**
 - 1.1 Core Design Principles
- 2. High-Level Cluster Architecture**
 - 2.1 Cluster Communication Flow
- 3. Control Plane Components**
 - 3.1 kube-apiserver
 - 3.2 etcd
 - 3.3 kube-scheduler
 - 3.4 kube-controller-manager
 - 3.5 cloud-controller-manager
- 4. Worker Node Architecture**
 - 4.1 kubelet
 - 4.1.1 kubelet Responsibilities
 - 4.1.2 kubelet Configuration
 - 4.2 kube-proxy
 - 4.3 Container Runtime
 - 4.4 Node Resource Management
- 5. Containers in Kubernetes**
 - 5.1 Container Fundamentals
 - 5.1.1 Linux Primitives Behind Containers
 - 5.2 Container Image Layers
 - 5.3 Container Lifecycle in Kubernetes
 - 5.3.1 Container States
 - 5.3.2 Container Probes
 - 5.4 Container Security Context
 - 5.5 Init Containers
- 6. Pods — The Fundamental Deployment Unit**
 - 6.1 Pod Anatomy
 - 6.1.1 Networking
 - 6.1.2 Storage
 - 6.2 Pod Lifecycle
 - 6.3 Pod Scheduling Controls
 - 6.3.1 Node Selection
 - 6.3.2 Pod Topology
 - 6.3.3 Taints and Tolerations
 - 6.4 Pod Controllers — Managing Pod Lifecycle at Scale
 - 6.5 Sidecar Containers
- 7. Storage Architecture — Storage Nodes & Persistent Volumes**
 - 7.1 What is a Storage Node?
 - 7.1.1 Storage Node Hardware Considerations
 - 7.2 The Kubernetes Storage Abstraction Model
 - 7.3 PersistentVolumes (PV)
 - 7.3.1 PV Access Modes
 - 7.3.2 PV Reclaim Policies
 - 7.4 PersistentVolumeClaims (PVC)
 - 7.5 StorageClasses & Dynamic Provisioning
 - 7.6 Container Storage Interface (CSI)
 - 7.6.1 CSI Driver Components
 - 7.7 StatefulSets and Persistent Storage
 - 7.8 Volume Snapshots & Data Protection
- 8. Kubernetes Networking Architecture**
 - 8.1 Services

8.2 Ingress & Ingress Controllers

8.3 DNS in Kubernetes

9. Kubernetes Security Architecture

9.1 Authentication & Authorization

9.2 Pod Security Standards

9.3 Network Policies

9.4 Secrets Management

10. Observability & Monitoring

10.1 Metrics

10.2 Logging

10.3 Horizontal Pod Autoscaler (HPA) & Vertical Pod Autoscaler (VPA)

11. Complete Architecture Summary

11.1 Key Relationships

12. Production Best Practices

12.1 Control Plane

12.2 Worker Nodes

12.3 Pods & Containers

12.4 Storage

13. Conclusion

1. Introduction to Kubernetes

Kubernetes (K8s) is an open-source container orchestration platform originally developed by Google and donated to the Cloud Native Computing Foundation (CNCF) in 2014. It automates the deployment, scaling, management, and networking of containerized applications across clusters of machines.

Kubernetes abstracts the underlying infrastructure and provides a unified API to manage workloads. Rather than manually placing containers on servers, engineers declare the desired state of their application, and Kubernetes continuously reconciles the actual state of the cluster toward that desired state. This self-healing, declarative model is the foundation of modern cloud-native architecture.

Why Kubernetes?

Traditional deployment methods require manual intervention for scaling, self-healing, and rolling updates. Kubernetes solves these challenges by providing an automated, declarative platform that treats infrastructure as code.

It enables organizations to run microservices at scale, manage hundreds of services across thousands of nodes, and maintain high availability without significant operational overhead.

1.1 Core Design Principles

- **Desired State Management:** Users declare what they want, and the system continuously works toward that goal.
- **Self-Healing:** Kubernetes restarts failed containers, replaces unhealthy pods, and reschedules workloads automatically.
- **Horizontal Scaling:** Applications can scale out by adding more pod replicas, not just vertically by using more resources.
- **Service Discovery & Load Balancing:** Built-in DNS and load balancing routes traffic to healthy instances.
- **Automated Rollouts & Rollbacks:** Changes to applications are rolled out gradually with automatic rollback on failure.
- **Secret & Config Management:** Sensitive data and configuration are managed separately from container images.

2. High-Level Cluster Architecture

A Kubernetes cluster is composed of two primary planes: the Control Plane (also called the Master) and the Data Plane (Worker Nodes). These two planes are distinct both in responsibility and in the components that run on each.

Plane	Responsibility	Primary Components
Control Plane	Manages cluster state, scheduling, and API access	kube-apiserver, etcd, kube-scheduler, kube-controller-manager, cloud-controller-manager
Data Plane (Worker Nodes)	Runs application workloads as containers inside Pods	kubelet, kube-proxy, Container Runtime (containerd/CRI-O)
Storage Layer	Provides persistent, stateful storage for workloads	PersistentVolumes, StorageClasses, CSI Drivers, Storage Nodes
Networking Layer	Enables pod-to-pod, pod-to-service communication	CNI Plugins (Calico, Flannel, Cilium), CoreDNS, Services

2.1 Cluster Communication Flow

All communication within a Kubernetes cluster flows through the API server. The kubelet on each Worker Node continuously sends heartbeats to the API server and watches for changes to Pods scheduled on that node. The scheduler watches for unscheduled Pods and assigns them to nodes. Controllers monitor cluster state and make corrective actions.

Key Insight: The API Server is the Brain

Every component in Kubernetes communicates exclusively through the kube-apiserver — never directly with each other. This design centralizes authentication, authorization, admission control, and audit logging. The API server is the single source of truth for cluster state, which is persisted in etcd.

3. Control Plane Components

The Control Plane is the set of components that manage the overall state of the cluster. It makes global decisions (such as scheduling) and detects and responds to cluster events (such as a pod going down). Control Plane components can run on any machine in the cluster but are typically isolated on dedicated nodes for stability and security.

3.1 kube-apiserver

The kube-apiserver is the front end of the Kubernetes control plane. It exposes the Kubernetes API over HTTPS and handles all REST operations. It is the only component that communicates with the etcd datastore. Every action in Kubernetes — from deploying a pod to scaling a deployment — passes through the API server.

- **Authentication & Authorization:** Validates user credentials via certificates, tokens, or webhook backends, and enforces RBAC/ABAC policies.
- **Admission Control:** Webhooks can mutate or validate incoming objects before they are persisted. Used for policy enforcement, sidecar injection, and more.
- **API Versioning:** Supports multiple concurrent API versions (v1, apps/v1, batch/v1) for backward compatibility.
- **Watch Mechanism:** Clients like kubelet and kube-scheduler use long-polling watches to receive real-time change notifications.

3.2 etcd

etcd is a distributed, strongly consistent key-value store used as Kubernetes' backing store for all cluster data. It stores the complete state of the cluster: all resource definitions, configuration, secrets, and status information. Without etcd, the cluster has no memory of its state.

etcd Availability & Consistency

etcd uses the Raft consensus algorithm. For a cluster with N nodes, it can tolerate $(N-1)/2$ node failures. Production clusters should run etcd with at least 3 nodes (tolerates 1 failure) or 5 nodes (tolerates 2 failures). etcd prioritizes consistency over availability — in a network partition, the minority partition stops serving writes to avoid split-brain.

- All data is stored under the /registry prefix in etcd.
- etcd supports TLS encryption for all client and peer communication.
- Regular etcd snapshots are essential for disaster recovery.
- etcd should be isolated on dedicated high-IOPS storage for production workloads.

3.3 kube-scheduler

The kube-scheduler watches for newly created Pods with no assigned node (unscheduled pods) and selects the best node for them to run on, based on a configurable set of policies and constraints.

Scheduling happens in two phases:

- **Filtering (Predicates):** Eliminates nodes that do not meet the pod's requirements. Checks include resource availability (CPU, memory), node selectors, taints and tolerations, affinity/anti-affinity rules, volume availability, and more.
- **Scoring (Priorities):** Ranks the remaining eligible nodes using priority functions. Factors include resource balance, node affinity preferences, pod topology spread, and custom priority plugins.

The scheduler is extensible via the Scheduling Framework — custom plugins can be written to implement bespoke scheduling logic without forking the scheduler binary.

3.4 kube-controller-manager

The kube-controller-manager runs a collection of controllers — each an independent control loop that watches the API server for changes and takes action to reconcile the actual state toward the desired state. Controllers are logically separate but compiled into a single binary for simplicity.

Controller	Responsibility
Node Controller	Monitors node health, marks nodes as Not Ready, evicts pods from failed nodes
Replication Controller	Ensures the specified number of pod replicas are running
Endpoints Controller	Populates the Endpoints object for Services (joining Services and Pods)
Service Account & Token Controllers	Creates default ServiceAccounts and API tokens for new namespaces
Deployment Controller	Manages rolling updates and rollbacks for Deployments
StatefulSet Controller	Manages ordered pod creation and stable identity for stateful workloads
DaemonSet Controller	Ensures a copy of a pod runs on every (or selected) node
Job & CronJob Controllers	Manages batch workloads and scheduled tasks
Horizontal Pod Autoscaler	Adjusts replica count based on CPU/memory/custom metrics

3.5 cloud-controller-manager

The cloud-controller-manager integrates Kubernetes with cloud provider APIs (AWS, GCP, Azure, etc.). It handles cloud-specific operations including load balancer provisioning, node lifecycle management based on cloud instance status, and persistent volume provisioning via cloud storage APIs. This separation allows the core Kubernetes codebase to remain cloud-agnostic.

4. Worker Node Architecture

Worker Nodes (also called Minion Nodes) are the machines in a Kubernetes cluster that actually run containerized workloads. Each Worker Node hosts one or more Pods, which contain the application containers. Worker Nodes can be physical or virtual machines and must run three core components: the kubelet, the kube-proxy, and a container runtime.

Worker Node Components at a Glance

kubelet — The node agent that communicates with the API server and manages containers on the node.

kube-proxy — Maintains network rules on the node for Services, enabling pod-to-service communication.

Container Runtime — The software responsible for running containers (containerd, CRI-O, Docker shim).

Container Network Interface (CNI) Plugin — Implements pod networking (Calico, Flannel, Cilium, Weave).

Node-level monitoring agents — (Optional) cAdvisor, Prometheus node exporter for metrics collection.

4.1 kubelet

The kubelet is the primary node agent. It runs on every Worker Node and is responsible for ensuring that the containers described in PodSpecs are running and healthy. The kubelet does NOT manage containers that were not created by Kubernetes.

4.1.1 kubelet Responsibilities

- **Pod Lifecycle Management:** Watches the API server for Pods assigned to its node and instructs the container runtime to start, stop, and restart containers accordingly.
- **Health Probing:** Executes liveness, readiness, and startup probes. Restarts containers that fail liveness probes. Removes pods from Service endpoints when readiness probes fail.
- **Resource Reporting:** Reports node resource usage and capacity to the API server. Updates node status conditions (Ready, OutOfDisk, MemoryPressure, DiskPressure, PIDPressure).
- **Volume Management:** Mounts and unmounts volumes as described in PodSpecs. Interfaces with the CSI driver for persistent storage.
- **Image Management:** Pulls container images from registries, manages local image cache, and enforces image pull policies.
- **Node Registration:** Registers the node with the API server on startup, advertising its resource capacity and labels.

4.1.2 kubelet Configuration

The kubelet is configured through a KubeletConfiguration object (YAML) or command-line flags. Key configuration options include resource reservations, eviction thresholds, TLS settings, and feature gates. The kubelet also interacts with the Container Runtime Interface (CRI) — a plugin API that decouples the kubelet from specific container runtimes.

4.2 kube-proxy

kube-proxy is a network proxy that runs on each Worker Node. It implements part of the Kubernetes Service concept by maintaining network rules (iptables, IPVS, or eBPF rules) that allow traffic to be correctly routed from Services to the backend Pods.

Mode	Mechanism	Performance	Notes
userspace	Packets routed through user-space proxy	Lowest	Legacy mode, rarely used in modern clusters
iptables	Kernel netfilter rules (default)	Good	Default mode; O(n) rule lookup with large number of services
IPVS	Kernel IP Virtual Server (load balancing)	Excellent	Supports multiple load-balancing algorithms, O(1) lookup
eBPF (Cilium)	Extended Berkeley Packet Filter programs	Best	Replaces kube-proxy entirely; requires Cilium CNI plugin

kube-proxy watches Services and Endpoints objects in the API server and updates its network rules accordingly. When a Service IP is hit, traffic is transparently load-balanced to one of the healthy backend Pod IPs.

4.3 Container Runtime

The container runtime is the software responsible for pulling images, creating containers, and managing their lifecycle at the OS level. Kubernetes communicates with the runtime via the Container Runtime Interface (CRI), a gRPC API defined by the kubelet.

Runtime	CRI Support	OCI Compliant	Description
containerd	Yes (native)	Yes	Industry standard; default runtime in most managed Kubernetes services (EKS, GKE, AKS)
CRI-O	Yes (native)	Yes	Lightweight runtime designed specifically for Kubernetes; used in OpenShift
Docker (via dockershim)	Removed in K8s 1.24	Yes	Docker daemon was never CRI-compliant; shim was removed; use containerd instead
gVisor (runsc)	Yes (via containerd)	Yes	Sandboxed runtime providing additional security via user-space kernel
Kata Containers	Yes (via containerd)	Yes	VM-based container isolation for strong multi-tenant security boundaries

4.4 Node Resource Management

Each node has a total capacity of CPU and memory. Kubernetes divides this capacity into allocatable resources and system/kubelet reserved resources. The scheduler only considers allocatable resources when placing pods.

Resource Allocation Formula

$\text{Allocatable} = \text{Total Capacity} - \text{System Reserved} - \text{Kube Reserved} - \text{Eviction Threshold}$

Pods specify resource requests (used by the scheduler for placement) and resource limits (enforced at runtime by cgroups). Pods without requests are considered 'best effort' and are the first to be evicted under memory pressure.

- **Guaranteed QoS:** requests == limits for all containers. Highest priority; last to be evicted.
- **Burstable QoS:** at least one container has a request or limit. Medium priority.
- **BestEffort QoS:** no requests or limits set. Lowest priority; first to be evicted.

5. Containers in Kubernetes

Containers are lightweight, isolated execution environments that package an application along with its runtime dependencies. In Kubernetes, containers are the atomic unit of execution — they cannot be scheduled individually but must always run inside a Pod.

5.1 Container Fundamentals

A container is built from a container image, which is a read-only, layered filesystem. Images are built using a Dockerfile (or Containerfile) and stored in container registries such as Docker Hub, Amazon ECR, Google Artifact Registry, or a private registry.

5.1.1 Linux Primitives Behind Containers

- **Namespaces:** Provide process isolation. Kubernetes uses PID, Network, Mount, UTS, IPC, and User namespaces to isolate containers from each other and the host.
- **cgroups (Control Groups):** Enforce resource limits for CPU, memory, disk I/O, and network bandwidth. Kubernetes sets cgroup limits based on pod resource requests and limits.
- **Union Filesystems:** Overlay filesystems (OverlayFS) stack image layers, allowing containers to share read-only base layers while each container has its own writable layer.
- **seccomp & AppArmor:** Security profiles that restrict system calls and file access available to containers, reducing the attack surface.

5.2 Container Image Layers

Container images are built as a series of read-only layers, where each instruction in a Dockerfile creates a new layer. Layers are cached and shared across images, making storage efficient. When a container runs, a thin writable layer is added on top — changes are written to this layer and discarded when the container stops.

Layer	Source	Writable?
Base OS layer (e.g., ubuntu:22.04)	FROM instruction	No
System dependencies layer	RUN apt-get install...	No
Application code layer	COPY ./app	No
Configuration layer	ENV / CMD / ENTRYPOINT	No
Container writable layer	Created at runtime	Yes (ephemeral)

5.3 Container Lifecycle in Kubernetes

Kubernetes manages the lifecycle of containers within Pods through a series of well-defined states and hooks that allow fine-grained control over startup, running, and shutdown behavior.

5.3.1 Container States

- **Waiting:** Container is not yet running. Reasons include image pulling, waiting for dependencies (init containers), or a CrashLoopBackOff backoff period.
- **Running:** Container is actively executing. The PostStart hook runs immediately after this state is entered.
- **Terminated:** Container has finished execution, either successfully (exit code 0) or with an error. The PreStop hook runs before termination.

5.3.2 Container Probes

Kubernetes uses probes to check the health of containers. These are critical for ensuring only healthy containers receive traffic.

Probe Type	Purpose	Failure Action
Liveness Probe	Is the container alive? Is it in a broken state?	Container is killed and restarted per the restartPolicy
Readiness Probe	Is the container ready to serve traffic?	Pod is removed from Service endpoints; no restart
Startup Probe	Has the container application started successfully?	Container killed if not succeeded within failureThreshold * periodSeconds

Each probe supports three mechanisms: exec (run a command inside the container), httpGet (perform an HTTP GET request), and tcpSocket (check if a TCP port is open).

5.4 Container Security Context

The securityContext field at the container level allows fine-grained security controls. These should always be configured in production to follow the principle of least privilege.

- **runAsNonRoot: true** — Prevents containers from running as root (UID 0). Essential security practice.
- **readOnlyRootFilesystem: true** — Mounts the container's root filesystem as read-only. Use emptyDir volumes for writable paths.
- **allowPrivilegeEscalation: false** — Prevents processes from gaining more privileges than their parent process.
- **capabilities** — Drop all Linux capabilities (drop: [ALL]) and only add specific required ones (e.g., NET_BIND_SERVICE).
- **seccompProfile** — Apply a seccomp filter to restrict available system calls. Use RuntimeDefault or a custom profile.

5.5 Init Containers

Init containers are specialized containers that run to completion before the main application containers start. They are defined separately from regular containers and run sequentially in the order they are defined. Each init container must complete successfully before the next one starts.

Init Container Use Cases

Waiting for external dependencies (databases, message brokers) to become available before starting the app.

Running database schema migrations before the application starts.

Cloning Git repositories or fetching configuration files into shared volumes.

Setting up required directory structures, permissions, or security certificates.

Registering the pod in a service registry or obtaining credentials from a secrets manager.

6. Pods — The Fundamental Deployment Unit

A Pod is the smallest and simplest unit in the Kubernetes object model that you create or deploy. A Pod represents a single instance of a running process in your cluster. Pods encapsulate one or more containers (application containers, init containers, ephemeral containers, and sidecar containers), storage resources, a unique network identity (IP address), and configuration options that govern how the containers should run.

The Pod Philosophy

Pods are designed to model an application-specific 'logical host'. Containers in a Pod share network and storage namespaces, making them tightly coupled. They can communicate with each other via localhost and can use shared volumes. This co-location is intentional for tightly related processes — for example, a web server and its log shipper sidecar.

Pods are ephemeral. They are not self-healing — if a Pod dies, it is replaced by a new Pod with a new UID. For resilience, you almost never create Pods directly; instead, you create higher-level controllers (Deployments, StatefulSets, DaemonSets) that manage Pods.

6.1 Pod Anatomy

6.1.1 Networking

Every Pod is assigned a unique cluster-wide IP address from the Pod CIDR range. All containers within a Pod share the same network namespace, meaning they share an IP address and port space. They communicate with each other via localhost. Pods communicate with other Pods directly using their IP addresses (the CNI plugin handles routing).

- Pod-to-Pod communication is flat — all pods can reach all other pods by IP, regardless of the node they are on.
- External traffic reaches Pods through Services, which provide stable virtual IPs and DNS names.
- Network policies can restrict which pods can communicate with which.

6.1.2 Storage

Pods can specify a set of Volumes. Containers in the pod can mount these volumes to specific paths in their filesystems. Volumes have lifetimes tied to the Pod (not individual containers), so data in a volume survives container restarts within the same Pod, but is lost when the Pod is deleted.

Volume Type	Scope	Use Case
emptyDir	Pod lifetime	Temporary scratch space; shared between containers in the same pod
hostPath	Node persistent	Accessing host filesystem; risky in production (tightly couples pod to node)
configMap / secret	Cluster	Inject configuration data or secrets as files or environment variables

Volume Type	Scope	Use Case
persistentVolumeClaim (PVC)	External storage	Durable storage that persists beyond pod lifetime, backed by a PersistentVolume
projected	Pod lifetime	Projects multiple sources (serviceAccountToken, configMap, secret) into a single volume
downwardAPI	Pod lifetime	Expose pod metadata (labels, annotations, resource limits) to containers as files

6.2 Pod Lifecycle

Phase	Meaning
Pending	Pod has been accepted by Kubernetes but containers are not yet running. Could be waiting for scheduling, image pulling, or volume binding.
Running	Pod has been bound to a node, all containers have been created, and at least one container is still running (or starting/restarting).
Succeeded	All containers in the Pod have terminated successfully with exit code 0, and will not be restarted.
Failed	All containers have terminated, and at least one terminated with a non-zero exit code or was killed by the system.
Unknown	Pod state could not be obtained. Typically means communication with the node hosting the pod has failed.

Container restart behavior is governed by the `restartPolicy` field: Always (default — always restart), OnFailure (restart only on non-zero exit), or Never (never restart). This policy applies to all containers in the Pod.

6.3 Pod Scheduling Controls

6.3.1 Node Selection

- **nodeSelector:** Simple key-value label matching to constrain which nodes a pod can run on.
- **nodeName:** Directly specify the name of the node to bypass the scheduler entirely.
- **nodeAffinity:** Rich expression-based node selection with required (hard) and preferred (soft) rules.

6.3.2 Pod Topology

- **podAffinity / podAntiAffinity:** Schedule pods near (or away from) other pods with matching labels. Used to co-locate related services or spread replicas across failure domains.
- **topologySpreadConstraints:** Control how pods are spread across topology domains (zones, regions, nodes) to achieve even distribution and high availability.

6.3.3 Taints and Tolerations

Taints are applied to nodes and prevent pods from being scheduled onto them unless the pod has a matching toleration. This is used to dedicate nodes for specific workloads (e.g., GPU nodes, high-memory nodes) or to mark nodes for maintenance.

Taint Effect	Behavior
NoSchedule	New pods without a matching toleration are not scheduled on the node. Existing pods are unaffected.
PreferNoSchedule	Kubernetes tries to avoid scheduling new pods without a toleration, but may do so if no other options exist.
NoExecute	New pods without a toleration are not scheduled. Existing pods without a toleration are evicted immediately (or after tolerationSeconds).

6.4 Pod Controllers — Managing Pod Lifecycle at Scale

In production, you rarely create Pods directly. Instead, you use higher-level controllers that manage sets of Pods and ensure the desired number of healthy replicas are always running.

Controller	Use Case	Key Feature
ReplicaSet	Maintains a stable set of replica Pods	Ensures exactly N replicas are running at all times
Deployment	Stateless applications with rolling updates	Manages ReplicaSets; supports rolling updates, rollbacks, and scaling
StatefulSet	Stateful applications (databases, caches)	Ordered startup/shutdown; stable network identity; stable persistent storage per pod
DaemonSet	Node-level services (monitoring, logging, networking)	Ensures exactly one pod per node (or selected nodes)
Job	One-time batch tasks	Runs pod to completion; retries on failure
CronJob	Scheduled batch tasks	Runs a Job on a cron schedule
ReplicationController	Legacy (replaced by ReplicaSet)	Older, less feature-rich version of ReplicaSet

6.5 Sidecar Containers

Kubernetes 1.29+ introduced native Sidecar Containers as a first-class concept (previously implemented as regular init containers). Sidecars run alongside the main application container and share the pod's network and storage. They are started before main containers and remain running for the pod's entire lifetime.

- Logging sidecars collect and forward logs from the main container's filesystem to a central log aggregator.

- Proxy sidecars (service mesh data planes like Envoy, Linkerd) handle mTLS, circuit breaking, and traffic management transparently.
- Configuration sidecars watch for configuration changes and reload the main application without restarting the pod.
- Secret management sidecars (e.g., Vault Agent) inject and renew secrets as files or environment variables.

7. Storage Architecture — Storage Nodes & Persistent Volumes

While stateless applications run perfectly on ephemeral pod storage, stateful workloads (databases, message brokers, file stores) require durable storage that persists beyond the lifecycle of any individual Pod. Kubernetes addresses this through a rich storage subsystem composed of PersistentVolumes, PersistentVolumeClaims, StorageClasses, and CSI drivers.

7.1 What is a Storage Node?

A Storage Node is a node in the Kubernetes cluster that is dedicated to or optimized for running stateful workloads that require high-performance or high-capacity persistent storage. Storage Nodes are typically configured with specialized hardware (NVMe SSDs, high-IOPS disks, large capacity HDDs) and are labeled/tainted to ensure that only storage-intensive workloads (such as database pods) are scheduled onto them.

Storage Node Configuration Pattern

Storage nodes are typically labeled (e.g., `node-role.kubernetes.io/storage=true`) and tainted (e.g., `dedicated=storage:NoSchedule`). Database pods (MySQL, PostgreSQL, Cassandra, Elasticsearch) are then configured with matching tolerations and nodeAffinity rules to be pinned to these nodes.

This ensures storage-intensive pods are co-located with their data, reducing network I/O, and that general-purpose pods do not compete for the node's I/O resources.

7.1.1 Storage Node Hardware Considerations

- NVMe SSDs for low-latency transactional databases (PostgreSQL, MySQL, etcd).
- High-throughput HDDs or object storage for large-scale analytics and archival.
- RAID configurations for data redundancy at the hardware level.
- Dedicated network interfaces (NICs) with high throughput for storage traffic isolation.
- NUMA-aware CPU configuration to minimize memory access latency for storage processing.

7.2 The Kubernetes Storage Abstraction Model

Kubernetes provides a layered storage model that decouples storage provisioning (handled by administrators) from storage consumption (handled by developers). This abstraction allows developers to request storage without needing to know the underlying infrastructure.

Layer	Object	Who Creates It	Purpose
Provisioning	StorageClass	Administrator	Defines the type of storage, provisioner, reclaim policy, and parameters
Provisioning	PersistentVolume (PV)	Admin or dynamic provisioner	Represents a piece of storage in the cluster with a

Layer	Object	Who Creates It	Purpose
			specific capacity and access mode
Consumption	PersistentVolumeClaim (PVC)	Developer / Application	A request for storage by a Pod, specifying size and access mode
Runtime	Volume Mount	Defined in Pod spec	Binds the PVC to a path inside a container

7.3 PersistentVolumes (PV)

A PersistentVolume is a piece of storage provisioned either statically (by an administrator) or dynamically (by a StorageClass provisioner). It has an independent lifecycle from any Pod and retains data even when no Pod is using it.

7.3.1 PV Access Modes

Access Mode	Short	Description	Example Use Case
ReadWriteOnce	RWO	Mounted as read-write by a single Node	Databases (MySQL, PostgreSQL)
ReadOnlyMany	ROX	Mounted as read-only by many Nodes simultaneously	Shared configuration or static assets
ReadWriteMany	RWX	Mounted as read-write by many Nodes simultaneously	Shared content stores, NFS-backed applications
ReadWriteOncePod	RWOP	Mounted as read-write by a single Pod (K8s 1.22+)	Strict single-pod write guarantees

7.3.2 PV Reclaim Policies

- **Retain:** PV is not deleted after PVC release. Data is preserved; manual reclamation required. Safest for production data.
- **Delete:** PV and underlying storage asset are deleted when the PVC is released. Used with dynamically provisioned volumes.
- **Recycle (deprecated):** Performs a basic scrub of the volume and makes it available for a new PVC. Deprecated in favor of dynamic provisioning.

7.4 PersistentVolumeClaims (PVC)

A PersistentVolumeClaim is a request for storage by a developer. It abstracts the details of storage provisioning — the developer simply specifies how much storage they need and what access mode is required. Kubernetes matches the PVC to an available PV (or dynamically provisions one via a StorageClass).

PVC binding is based on matching capacity (PV must be $\geq$ PVC request), access modes, volume mode (Filesystem or Block), and optionally a StorageClassName or label selectors. Once bound, a PVC-PV pair has a one-to-one relationship.

7.5 StorageClasses & Dynamic Provisioning

StorageClasses enable dynamic provisioning of PersistentVolumes on demand, eliminating the need for administrators to pre-provision storage. When a PVC requests a StorageClass, the associated provisioner automatically creates a PV that satisfies the request.

Provisioner	Cloud / Platform	Storage Type
kubernetes.io/aws-ebs	Amazon Web Services	EBS (Elastic Block Store) — block storage
pd.csi.storage.gke.io	Google Cloud Platform	Persistent Disk — block storage
disk.csi.azure.com	Microsoft Azure	Azure Managed Disks — block storage
file.csi.azure.com	Microsoft Azure	Azure Files — shared (NFS/SMB) storage
efs.csi.aws.com	Amazon Web Services	EFS (Elastic File System) — shared NFS storage
rook-ceph.rbd.csi.ceph.com	On-premises / Hybrid	Ceph RBD — block storage
rook-ceph.cephfs.csi.ceph.com	On-premises / Hybrid	CephFS — shared filesystem
openebs.io/local	On-premises	OpenEBS Local PV — node-local storage
longhorn.io	On-premises	Longhorn — distributed block storage

7.6 Container Storage Interface (CSI)

The Container Storage Interface (CSI) is a standardized API between container orchestrators (like Kubernetes) and storage providers. CSI drivers are distributed independently from Kubernetes and allow storage vendors to build plugins without modifying Kubernetes core code.

7.6.1 CSI Driver Components

- **CSI Controller Plugin:** Runs as a Deployment. Handles volume lifecycle operations: CreateVolume, DeleteVolume, ControllerPublishVolume, CreateSnapshot, ListSnapshots.
- **CSI Node Plugin:** Runs as a DaemonSet on every node. Handles NodeStageVolume (mount to global path), NodePublishVolume (bind mount to pod path), and NodeUnpublishVolume.
- **External Provisioner sidecar:** Watches PVCs and triggers CreateVolume CSI calls. Runs alongside the CSI Controller.
- **External Attacher sidecar:** Manages VolumeAttachment objects and calls ControllerPublishVolume/ControllerUnpublishVolume.

- **External Resizer & Snapshotter:** Handle volume expansion and snapshot lifecycle respectively.

7.7 StatefulSets and Persistent Storage

StatefulSets are the primary controller for running stateful workloads. They provide three guarantees crucial for databases and stateful services: stable, unique network identifiers (Pod-0, Pod-1, etc.); stable persistent storage (each pod gets its own PVC that persists through pod rescheduling); and ordered, graceful deployment and scaling.

StatefulSet Volume Claim Templates

StatefulSets use volumeClaimTemplates to automatically create a PVC for each pod replica. For example, a StatefulSet with 3 replicas and a volumeClaimTemplate named 'data' will create PVCs: data-podname-0, data-podname-1, data-podname-2.

Critically, when a StatefulSet pod is rescheduled to a different node, Kubernetes reattaches the same PVC to the new pod, ensuring data persistence across pod restarts and node failures.

7.8 Volume Snapshots & Data Protection

Kubernetes supports VolumeSnapshots, VolumeSnapshotContents, and VolumeSnapshotClasses — objects that mirror the PV/PVC/StorageClass hierarchy for snapshot management. Snapshots can be used to create point-in-time backups and to clone volumes for test environments.

- **VolumeSnapshot:** A namespaced request by a user for a snapshot of an existing PVC.
- **VolumeSnapshotContent:** A cluster-scoped resource representing the actual snapshot, similar to a PV.
- **VolumeSnapshotClass:** Defines snapshot parameters and the CSI driver to use, similar to StorageClass.
- **DataSourceRef on PVC:** A new PVC can be pre-populated from an existing VolumeSnapshot.

8. Kubernetes Networking Architecture

Kubernetes has a well-defined networking model with four distinct communication problems it addresses: container-to-container within the same pod, pod-to-pod across nodes, pod-to-service, and external-to-service. Each is handled by a different mechanism.

8.1 Services

A Service is an abstraction that defines a logical set of Pods and a policy for accessing them. Services provide a stable IP and DNS name in front of a dynamic set of Pods. Traffic is load-balanced across healthy pods matching the Service's selector.

Service Type	Accessibility	Use Case
ClusterIP (default)	Internal cluster only	Internal microservice communication; not accessible from outside the cluster
NodePort	External via node IP:port	Development/testing; exposes service on a static port (30000-32767) on every node
LoadBalancer	External via cloud LB	Production; provisions a cloud load balancer with a dedicated external IP
ExternalName	DNS CNAME redirect	Abstract external services (e.g., cloud databases) behind a Kubernetes DNS name
Headless (clusterIP: None)	Direct pod IPs via DNS	StatefulSets; direct pod addressing; used by databases for client-side load balancing

8.2 Ingress & Ingress Controllers

Ingress is a cluster-scoped resource that manages HTTP and HTTPS access to Services from outside the cluster. It provides URL-based routing, SSL/TLS termination, name-based virtual hosting, and load balancing. An Ingress Controller must be deployed in the cluster to implement the Ingress resource — common choices include NGINX Ingress Controller, Traefik, and the AWS ALB Ingress Controller.

8.3 DNS in Kubernetes

CoreDNS runs as a Deployment in the kube-system namespace and provides DNS-based service discovery for the cluster. Every Service and (optionally) Pod gets a DNS name in the format: service-name.namespace.svc.cluster.local. Pods can also get A records if dnsPolicy: ClusterFirstWithHostNet or pod IP DNS is enabled.

9. Kubernetes Security Architecture

Security in Kubernetes is a multi-layered concern that spans the entire stack from the cluster control plane down to individual container processes. The Kubernetes security model is often described using the 4Cs framework: Cloud, Cluster, Container, Code.

9.1 Authentication & Authorization

- **Authentication:** Kubernetes supports X.509 client certificates, Bearer tokens, OpenID Connect, and webhook token authentication. Service accounts use auto-mounted JWT tokens for pod authentication.
- **RBAC (Role-Based Access Control):** Roles define permissions on resources (verbs: get, list, create, delete). RoleBindings associate roles with users, groups, or service accounts. ClusterRoles/ClusterRoleBindings apply cluster-wide.
- **Admission Control:** Webhooks (MutatingAdmissionWebhook, ValidatingAdmissionWebhook) enforce policies, inject sidecars, and validate resource configurations before persistence.

9.2 Pod Security Standards

Kubernetes 1.25 removed PodSecurityPolicy and replaced it with Pod Security Standards, enforced by the Pod Security Admission controller. Three predefined profiles exist:

Profile	Restriction Level	Use Case
Privileged	No restrictions	System-level and infrastructure workloads (CNI, CSI drivers)
Baseline	Minimal restrictions; prevents known privilege escalations	General-purpose non-critical applications
Restricted	Heavily restricted; follows pod hardening best practices	Security-sensitive applications; recommended for all production workloads

9.3 Network Policies

NetworkPolicy resources define ingress and egress rules for Pods at the IP/port level. By default, all pods can communicate with all other pods. Network Policies (enforced by the CNI plugin) can restrict this to implement zero-trust networking within the cluster. CNI plugins that support Network Policies include Calico, Cilium, and Weave Net.

9.4 Secrets Management

Kubernetes Secrets store sensitive data (passwords, tokens, certificates) separately from pod specs. Secrets are base64-encoded (not encrypted by default) in etcd. Best practices include enabling encryption at rest for etcd, using external secret managers (HashiCorp Vault, AWS Secrets Manager, GCP Secret Manager) via the Secrets Store CSI Driver, and restricting secret access via RBAC.

10. Observability & Monitoring

A production Kubernetes cluster requires comprehensive observability across metrics, logs, and traces. The Kubernetes ecosystem has converged on the Prometheus/Grafana stack for metrics, Fluentd/Fluent Bit/Loki for logging, and Jaeger/Tempo for distributed tracing.

10.1 Metrics

- kube-state-metrics: Exposes Kubernetes object state as Prometheus metrics (pod status, deployment replicas, PVC bindings).
- metrics-server: Provides real-time CPU/memory metrics for the `kubectl top` command and Horizontal Pod Autoscaler (HPA).
- Prometheus: Scrapes metrics from pods, nodes, and Kubernetes components. Stored in a time-series database.
- Grafana: Visualizes Prometheus metrics with pre-built Kubernetes dashboards (USE method, RED method, node utilization).

10.2 Logging

- Node-level logging: kubelet logs, container runtime logs, kernel logs — critical for diagnosing node issues.
- Application logging: Containers write to `stdout/stderr`; kubelet captures these via the container runtime.
- Cluster-level logging: DaemonSet-based log agents (Fluent Bit, Fluentd) collect logs from all nodes and forward to a central store (Elasticsearch, Loki, CloudWatch).

10.3 Horizontal Pod Autoscaler (HPA) & Vertical Pod Autoscaler (VPA)

HPA automatically scales the number of pod replicas based on observed CPU/memory utilization or custom metrics from Prometheus via the Custom Metrics API. VPA automatically adjusts pod resource requests and limits based on historical usage, right-sizing workloads over time.

11. Complete Architecture Summary

The following table provides a consolidated reference of all major Kubernetes components, their location, and their primary function.

Component	Location	Primary Function
kube-apiserver	Control Plane	Central API hub; all communication passes through; enforces auth, admission
etcd	Control Plane	Distributed key-value store; sole persistent state store for the cluster
kube-scheduler	Control Plane	Assigns unscheduled Pods to nodes based on resource and policy constraints
kube-controller-manager	Control Plane	Runs all built-in control loops to reconcile desired vs actual state
cloud-controller-manager	Control Plane	Integrates with cloud provider APIs for LBs, node lifecycle, storage
kubelet	Worker Node	Node agent; manages pod/container lifecycle; reports node status
kube-proxy	Worker Node	Maintains network rules for Service routing; iptables/IPVS/eBPF
Container Runtime	Worker Node	Pulls images and runs containers (containerd, CRI-O)
CNI Plugin	Worker Node	Implements pod networking; assigns IPs; enforces NetworkPolicies
Pod	Worker Node	Smallest deployable unit; encapsulates 1+ containers, networking, storage
Container	Inside Pod	Lightweight process isolation using Linux namespaces and cgroups
PersistentVolume (PV)	Cluster-scoped	Represents a piece of durable storage in the cluster
PersistentVolumeClaim (PVC)	Namespace-scoped	Developer's request for persistent storage; bound to a PV
StorageClass	Cluster-scoped	Defines storage type and provisioner for dynamic PV creation
CSI Driver	Cluster (Controller) + Node (DaemonSet)	Plugin interface between Kubernetes and external storage systems
Storage Node	Dedicated Worker Node	Specialized node for stateful workloads with optimized storage hardware
CoreDNS	kube-system namespace	Cluster DNS; resolves Service and Pod names to IP addresses

Component	Location	Primary Function
Ingress Controller	Namespace (typically)	Implements Ingress rules; HTTP(S) routing and TLS termination

11.1 Key Relationships

Understanding how components relate to each other is critical for operational proficiency:

- A Deployment manages ReplicaSets, which manage Pods.
- A StatefulSet manages Pods and creates PVCs from volumeClaimTemplates; each Pod is bound to its own PVC.
- A DaemonSet ensures one Pod runs per node — used for CNI plugins, log agents, monitoring agents, and CSI node drivers.
- A Service selects Pods via label selectors; kube-proxy implements the routing rules.
- An Ingress routes HTTP traffic to Services; an Ingress Controller implements the Ingress rules.
- PVCs are bound to PVs; PVs are backed by CSI-provisioned storage or statically created volumes.
- The kubelet on each node reports to the API server; the scheduler and controllers act based on API server state.

12. Production Best Practices

12.1 Control Plane

- Run the control plane on dedicated nodes with no user workloads.
- Run etcd with at least 3 nodes on high-IOPS SSDs; take regular snapshots.
- Enable etcd encryption at rest using a KMS provider.
- Use RBAC with least-privilege principles; avoid cluster-admin bindings.
- Enable audit logging on the API server for compliance and forensics.

12.2 Worker Nodes

- Use node pools with homogeneous hardware; separate node pools for different workload types.
- Set kernel parameters (inotify watches, file descriptors, network buffers) for containerized workloads.
- Use containerd as the container runtime; keep it and the kernel updated.
- Configure kubelet with appropriate resource reservations for system and kube components.
- Enable node-level security with CIS Kubernetes Benchmark and OS hardening.

12.3 Pods & Containers

- Always set resource requests and limits; avoid BestEffort QoS in production.
- Run containers as non-root with readOnlyRootFilesystem and minimal capabilities.
- Use multi-stage Docker builds to minimize image size and attack surface.
- Configure liveness, readiness, and startup probes for all production containers.
- Never store secrets in environment variables; use volume-mounted secrets or a CSI secrets driver.

12.4 Storage

- Use dynamic provisioning with StorageClasses; avoid manual PV management at scale.
- Use the Retain reclaim policy for production databases to prevent accidental data deletion.
- Implement regular VolumeSnapshot-based backups for all stateful workloads.
- Size PVCs generously; use volume expansion capability rather than over-provisioning initially.
- Co-locate storage nodes with stateful pods using node affinity and taints for I/O performance.

13. Conclusion

Kubernetes has become the de facto standard for container orchestration because it solves the fundamental challenge of running distributed applications reliably at scale. Its architecture — centered around a declarative API, a self-healing control plane, extensible worker nodes, and a powerful storage abstraction — provides the foundation for building highly available, scalable, and secure cloud-native applications.

Understanding the interplay between the Control Plane, Worker Nodes, Containers, Pods, and the Storage subsystem is essential for operating Kubernetes effectively. The Control Plane governs intent; Worker Nodes execute workloads; Pods are the atomic unit of deployment; Containers provide the isolated runtime; and Storage Nodes with PVs/PVCs/CSI drivers ensure that stateful workloads retain their data reliably across the dynamic lifecycle of the cluster.

The Path Forward

As Kubernetes continues to evolve, key areas include: eBPF-based networking and observability (Cilium, Tetragon), Gateway API replacing Ingress, Cluster API for declarative cluster management, multi-cluster federation, and AI/ML workloads using GPU operator and MPI operator.

Mastery of the foundational architecture presented in this document will serve as the basis for navigating these advanced topics and building production-grade Kubernetes platforms.

— End of Document —

© Tech with Vishal Abhinav | Platform Ops Engineer

All rights reserved. This document is authored and published by Tech with Vishal Abhinav.